

Name :- Sawant Aarush Hemant

Class :- T.Y.B.Sc.(I.T.)

Roll No :- 269729

Subject :- DevOps (Assi.) Sem :- Vth

Index

Gr. No.	Date	Topic	Sign
1.	04/07/2026	DevOps : Conflict to Collaboration	Jyoti 4/7
2.	23/07/2026	Essential DevOps Tooling	Jyoti 23/7
3.	06/08/2026	DevOps Adoption Plays	Jyoti 6/8
4.		Tuning & Continuous Improvement	
5.		Scaling & Leading DevOps for the Enterprise	

[illegible]

Sr. No.	Date	Topic	Sign
1	4/7/26	DevOps: Conflict to Collaboration	Trisha

Q. 1. How does DevOps help achieve balance betⁿ innovation & stability & betⁿ speed of delivery & quality

-
1. DevOps starts from recognition that innovation versus stability & speed of delivery v/s quality, only took like trade-offs when Dev & Ops work in silos.
 2. Historically, development teams were measured on the velocity of change, while IT operations carried the mandate of "resilience & stability at all costs, even if that means holding back change."
 3. This structural friction is precisely what DevOps for Digital Leaders call the IT Impasse - "Faster Software Development v/s Operational Stability."
 4. Serial, stop-start handoffs (Dev codes, then QA tests, then Ops monitors) could not keep pace with software now released in weeks/days instead of years.

DevOps resolves this tension rather than choosing a side

- Empirical evidence: 2016 Puppet Labs 'State of DevOps' report found that high-performing IT Org. achieve faster software delivery along with increased resilience and stability simultaneously - proving two goals are not mutually exclusive once good DevOps practices are in place.

- Cultural shift - Systems thinking: DevOps culture replaces functional "turf wars" with a collective focus on product/service as a whole rather than discrete silos. "Maverick" behaviour - cutting corners/blaming Ops when a release fails - is countered by a 'unified IT is the hero' mindset. So quality & stability become shared, not adversarial goals.

- Pipeline Automation: CI & CD automate build, test & deployment, which simultaneously reduces human error (raising quality) & shortens release cycle (raising speed).

- Small batch sizes & shift-left testing: Delivering smaller, more frequent changes reduces the blast radius & risk of each other release while allowing continuous innovation; testing earlier in lifecycle catches defects when they're cheaper to fix, improving quality without slowing throughput.

- Lean waste elimination: By removing constraints (e.g. lack of test data, manual hand-offs) that cause both delay & defects, DevOps improves speed & quality together instead of sacrificing one for other.

In short, DevOps treats innovation/stability & speed/quality as two ends of same continuum: cultural collab & automation deliver more often & reliably, thus time

Q.2

Explain Continuous Integration & Delivery practices
Continuous Integration (CI) & Continuous Delivery (CD) are described as two essential capabilities at core of DevOps - "Without these, No DevOps." Both exist to reduce cycle time, time from inception of a need to a tested, deployable capability.

CI is Agile practice of integrating each developer's work with rest of team on a regular cadence & testing integrated build immediately instead of treating integration as costly, risky activity left until end of development (waterfall). Martin Fowler 10 CI practices are central to this:

- Maintain a single-source repo for all code/artifacts
- Automate build so integration happens continuously
- Make build self-testing with automated unit & component & app-level tests.
- Ensure everyone commits to mainline every day to keep integrations simple.
- Ensure every commit builds mainline on an integration machine.
- Keep the build fast so it never becomes a bottleneck
- Test in a clone of production environment for realistic results.
- Make it easy for anyone to get latest executable for validation.
- Make sure everyone can see what is happening (visibility via dashboard/portals).
- Automate deployment - the practice that code directly into CD.

CD takes output of every CI build & progresses it automatically through rest of delivery pipeline (Dev → Build → Package → Deploy → Test Stage → Production) so that app. is always in a deployable state. Only "good" builds move to QA & only QA-validated, production-ready builds move to staging/production. Key clarifications.

- CD isn't same as continuous deployment - it does not mean every change is automatically pushed to production. It's the capability to deploy to any environment, at any time, as needed.
- What is delivered may be full app, one or more components, content or config/env. changes not necessarily a complete release.
- CD needs orchestrating code, content, middleware & env. config. together (not just "moving files around").
- Two of Fowler's ten CI practices - testing in a production-like clone & automating deployment - are technical bridge that makes CD possible.

Both form a continuous delivery pipeline that turns every commit into a reliable release candidate, providing fast feedback that drives continuous improvement of software, the environment & the delivery process itself.

Q 3. How does DevOps serve as a key component in achieving business agility?

→ Industry research cited in DevOps for Digital Leaders shows 80% org. surveyed view DevOps as a "key component of business agility" — though only 55% had yet built a well-defined DevOps strategy, underlining how central, yet still maturing, this link is.

1. Faster idea-to-market translation: DevOps compresses cycle from a new idea to "code running in production", letting org. respond to changing customer needs far faster than traditional siloed SDLCs. The ING Bank case illustrates this: moving from slow, infrequent releases to CD took its mobile app from a 1-star to 4-star rating.

2. Levelling field with startups: Large enterprises traditionally "move toward stability", limiting growth, while unburdened startups (e.g., eBay, Airbnb) pivot quickly & displace incumbents (Sears, Hilton). DevOps gives large org. same "lean & mean" agility of a startup without losing enterprise scale.

3. Continuous value creation: As Jez Humble states "long-term value of an enterprise is its ability to continuously increase value it gives to customers through innovation." Agility, in this sense, is an ongoing capability, not a one-time project.

4. lean lineage : DevOps inherits disruption - through - technological - innovation, Tenets of Toyota Way, showing that tech-driven agility has long been a competitive differentiator, not just an IT concern.

5. Operational enablers : The same DevOps capabilities - CI/CD, automation, cross-functional collabor - shorter feedback loop betⁿ business idea, technical execution & customer response which is practical essence of agility.

DevOps operationalises business agility by collapsing time, cost & risk of delivering change & enabling orgs of any size & history to behave like an agile startup while retaining scale & reach of an enterprise.

Q.4 Distinguish betⁿ monolithic & microservice app architectures.

→ To support continuous innovation at velocity, orgs are supplementing older monolithic designs with microservice architectures.

Monolithic Architecture :

All app. functionality - UI, business logic, data access - is packaged & operated as a single unit, scaled by deploying entire app. on more servers. Although it may internally consist of many services, it behaves as one.

indivisible hole. Challenges:

1. Brittleness: If a single component fails / consumes excess resources, entire app. can fail / degrade.
2. Risk: $\therefore$ everything is packaged & fails together, teams hesitate to change the stack causing resistance to frequent deployment.
3. Tightly coupled - scaling means deploying whole app, which is problematic for unpredictable demand (e.g., mobile apps).
4. Dependencies - developers cannot easily work, test / deploy independently, increasing development time.

Microservice Architecture

The app. is decomposed into a set of small, independent services, each focused on one specific business capability (e.g., login, shopping cart, payments, recommendations), loosely coupled & communicating via APIs. Each microservice typically has its own DB and can be built, tested, deployed & scaled independently, by cross-functional teams organised around business capabilities. Benefits:-

1. Independent Deployment: Services scale individually to meet demand spikes.
2. Independent Coding: Teams can use different languages / frameworks but suited to each

service, avoiding lock-in to one stack.

3. Fault tolerance: Failure of one microservice (e.g., recommendations) doesn't bring down the whole system (e.g., checkout can still work).

4. Increased agility: Independent build/test/deploy cycles better support CD.

Q. 5. Identify 8 elements of waste. What are some immediate practices that cross-functional teams can apply to help identify & eliminate waste?

→ Applying lean thinking to software delivery. DevOps for Digital Leaders identifies 8 elements of waste, or "Muda", using the mnemonic D.O.W.N.T.T.M.E.:

1. Defects: Badly designed/poor-quality code, non-functional performance issues → lost customers, revenue & brand damage.

2. Overproduction: Delivering unwanted features or producing excess capacity for unanticipated demand → delays, cost overrun.

3. Waiting: Release backlogs/bottlenecks, unavailable test infrastructure/data, lengthy change/security reviews → slow time to market.

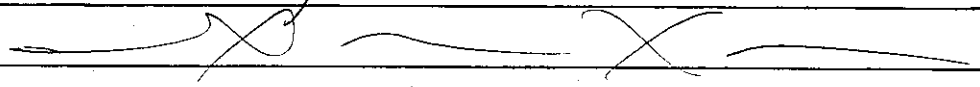
4. Non-value added processing: Lengthy fire-fighting, team-level priorities over program-level objectives → morale issue, high turnover.
5. Transportation: Frequent release rollbacks, Dev & QA hand-offs → launch delays, increased cycle times.
6. Inventory (excess): Underutilized resources, excessive work-in-progress → increased capital/operational cost.
7. Motion: Devs constantly task-switching, relearning & rework → lost productivity, talent erosion.
8. Employee knowledge (unused): Closed retrospective/stand-ups, no feedback loop from service desk → missed improvement opportunities.

These elements are interlinked: undetected defects can trigger extra hardware purchases (inventory), which increases support burden (non-value-added processing) waste begets waste & unmanaged technical debt accumulates.

Immediate practices to identify & eliminate waste:

pto →

- 1) Prevent defects by removing constraints
- 2) Focus on value to prevent overproduction
- 3) Smooth flow to reduce wait times.
- 4) Limit non-value-added processing with data-driven insight.
- 5) Reduce transportation cost by automating deployments
- 6) Eliminate excess inventory
- 7) Prevent unnecessary motion with parallel development.



Sr. No.	Date	Topic	Sign
2	23/7/26	Essential DevOps Tooling	Juch

Q.1. Explain tactical initiatives org use to assess API mgmt. capabilities.

→ As legacy monolithic systems are supplemented with microservices (each fronted by an API), effective API mgmt. becomes essential. Org assess their API mgmt. capabilities by reviewing strategic API objectives against 4 tactical initiatives (plays):

1) Create & Integrate APIs.

Moves towards scalable, sustainable API creation that unlocks value from legacy app. & data stores. Capabilities needed: connecting SOA/ESB/legacy sys. (protocol adaptation, mediation transformation); aggregating data from multiple SQL/NoSQL sources with fine-grained access control; connecting cloud services & generating enterprise-grade REST APIs from reactive, logic-based processing.

2) Secure the Open Enterprise

Builds security into API lifecycle from design rather than bolting it on afterward; protection against OWASP threats (SQL injection, cross-site scripting, denial-of-service); controlled access via SSO & identity mgmt. & end-to-end security.

3) Unlock Business Value of Data
provides simplified, controlled, compliant access to data (authorization, shaping, policy mgmt.) supports a wider partner/public developer ecosystem (discovery, acquisition, design, collaboration) & reduces mobile app. deliver time using reusable SDKs/APIs for security, messaging & offline storage.

4) Accelerate Mobile & To-T Development

Positions API mgmt. as the launch pt. for digital strategy: monetising APIs & building digital ecosystems with control, compliance, security & reporting needed to expand across the platforms, devices & partners.

Q. 2. What are new skills testing professionals need fully leverage as advanced automation becomes more pervasive?

→ As automation becomes central to CI & CD, testing shifts from a transactional, end-of-cycle "gate" fn to a consultative discipline embedded across the SDLC. QA & testing professionals need to develop these new skills:

1. Thinking beyond pass/fail: helping business understand actual customer experience & supplying realistic data conditions to simulate it during testing.

- 2) Intimate understanding: Given growing app. complexity, testers must deeply understand all dependencies & be able to identify & remove constraints.
- 3) Assurance to analytics: Moving from simply documenting defects, to collecting & aggregating data from automated tests to determine root causes & where more rigorous testing is needed.
- 4) Early & thorough testing: Incorporating testing into acceptance criteria at very start of sprint, with QA, security & op. all reviewing before development proceeds.
- 5) Mentorship over conflict: Working closely & collaboratively with devs to improve resilience of dev-written tests, instead of being called in only when fragile tests fail.

Q. 3. Explain obstacles to CD.

→ Development Challenges:

- 1) Manual, error-prone, time-wasting environment provisioning & release processes.
- 2) Numerous errors throughout release cycle needing extensive detective work to trace root causes.
- 3) Slow response to customer feedback & market needs hurting customer retention & an initiative

Operations Challenges:

With an emphasis on stability, major obstacle concern resilience as deployment volume & velocity increase:

- 1) Fractured release processes managed via spreadsheets, scripts & disparate tools.
- 2) Long weekends, low morale & stress caused by problematic production deployments.
- 3) Loss of customers & revenue from downtime, outages or deployment errors.

Q.4 Explain CD maturity levels.

→ Level 1: Manual: Success depends on heroics & competence of individuals. Teams operate in silos; releases are error-prone & infrequent.

Level 2: Scripting: Deployment is planned per release & tracked; some automation exists, with version control & automated builds e.g. Nexus, Jenkins, Chef, Puppet.

Level 3: Automated: Common; Repeatable, automated delivery processes exist across environments & releases; release processes, metadata & artifacts are tracked under full lifecycle control.

Level 4: Continuous: Release automation orchestrates the entire release promotion process enabling predictable, monitored, measurable CD from development to production.

Level 5: Optimized: All elements work in a fully orchestrated fashion; achieving zero-touch deployment from planning to production. Feedback loops, deployment-pattern analysis & scenario simulation continuously improve cost-performance; teams manage multiple releases & their lifecycle delivery & realization.

Q.5. What are new skills agile operations teams need to acquire

→ IT ops. is increasingly judged not on how well it fixes problems, but on its contribution to business outcomes - working collaboratively with development to build quality in proactively.

1) Proactive Engagement:

Move from "console watching" & break-fix work toward intimately understanding app. behaviour, nurturing production systems & feeding info - & knowledge back into delivery.

2) Designing for Failure

Accepting that with cloud apps, & many moving parts failures are inevitable; objective shifts from preventing failure to designing systems & processes so that failures are contained & the business keeps running.

3) Moving Beyond Resilience

With complex, distributed microservices, resilience alone isn't enough - systems must be "Hydra-like" continuously improved both technically & commercially after every significant event, +ve or -ve.

4) Making Support a Top Design Issue.

Engaging upstream with devs in advisory capacity rather than as downstream "traffic cops" - sharing in wear crash analytics & APM data as devs

can design more supportable systems & jointly conducting triage exercises.

5) Active Monitoring

Moving beyond simple severity-based alert baseline to actively analysing both severity & duration of issues, removing false fire/noise & combining active alerting with deeper diagnostic capture.

Sr. No	Date	Topic	Sign
3	06/8/26	DevOps Adoption Plays	Pruthi

Q.1- Explain two areas that any team developing & executing a DevOps transformation roadmap needs to tackle.
 → Two areas of preparation:-

1) Addressing the Productivity Dip

Introducing any change - to processes, tools or team structure - causes an immediate, unavoidable drop in productivity before any gain is realised as people move from mastery of old way through a learning curve toward mastery of new way. A transformation succeeds only if productivity lost during dip is small relative to productivity gained afterward; many projects never recover if dip becomes too deep. To minimize dip; use experienced DevOps coaches embedded with project teams.

2) Overcoming Cultural Inertia

Even with full process & tooling improvement, DevOps adoption fails unless an org. overcomes its inherent resistance to change. Large orgs. accumulate inertia over yrs - symptoms include phrases like "this is way we do things here" along with governance checks, unused reports, & approval needs that exist "just because" without anyone questioning their value. Overcoming this needs a serious, honest review of every artifact & process that makes org. inefficient, since cultural transformation must be continually reinforced.

Q. 2. Explain in brief components of Business Model Canvas.

→ To build a business case for DevOps transformation, playbook uses this Canvas, contains 9 components that define, refine & assess viability of a business model:

- Customer Segments - Who are the customers being served?
- Value Proposition - What value is delivered to solve each customer segment's problem?
- Channels - What modes are used to deliver the value propositions to customers?
- Customer Relationships - What kind of relationship is established with each customer segment?
- Revenue Streams - What revenue sources result from each value proposition?
- Key Resources - What resources are needed to deliver the value propositions?
- Key Activities - What activities must be performed using those resources, to deliver value propositions?
- Cost Structures - What costs & investments are needed to deliver value propositions?

Q. 3. Explain two categories of apps. in an org. based on business intent. Give characteristics of these two types.

→ Which DevOps play to run depends not just on the bottleneck being addressed but on core business intent behind app. Based on business intent, app. fall into two categories - a better classification

than older "two-speed" or "bimodal" models, since business value delivered is a more meaningful measure than either speed or how often needs change.

1. Industrialized Core (Business Intent: Optimization)
These are core systems that "keep the business going" - well-understood, business models & user engagement methods. Characteristics:

- Success measured by stability & uptime.
- Typically large, complex & monolithic in architecture given their age.
- Business models/user engagement are well understood, even as specific needs evolve.
- Delivered at a steady, well-defined cadence matching business' capacity to absorb change.
- Lower risk tolerance - optimization is goal.

2. Innovation Edge (Business Intent: Innovation)
These apps. drive growth by experimenting with new business models, markets & user engagement methods using lean-startup techniques like MVP, A/B testing & design thinking. Characteristics:

- Measured by speed of delivering changes in response to user feedback.
- Typically small, built on modern tech. & platforms.
- Higher risk tolerance - failure is accepted as part of experimentation process.

Q.4. Define delivery cycle time. Explain various other cycle times embedded within delivery cycle time.

→ Delivery Cycle Time is defined as time it takes for each software component - whether a new component, an enhanced version of an existing one or a bug-fix - to go from inception to code running in production & for user feedback on it to return to team. Continuous delivery results in continuous feedback, which drives continuous improvement of software environment it runs in & process by which it was delivered; the goal of DevOps is to make this feedback loop as short as possible.

Delivery cycle time has several other, more granular cycle times embedded within it & reducing overall delivery cycle time can only be achieved by reducing each of these:

- Dev-test cycle time - Time for devs to get test results back from test team for a newly delivered component.
- Deployment cycle time - Time to deploy an app to an environment & begin using it for testing or production.
- Ops cycle time - Time for Ops to receive, provision & make available a new environment request.

- Datacenter latency cycle time - Time to get a ping response from a remote datacenter.
- Project approval cycle time - Time for approval board/committees to approve a new project.
- Change mgmt. cycle time - Time for Change Control Board to approve a change.

Q: 5 Explain in brief benefits of reducing batch size.

→ Lean thinking (Reinertsen) focuses on three areas: minimizing inventory, explicitly managing backlogs, & matching batch size to team capacity. While Agile addresses the first two through groomed backlogs & sprints, DevOps focuses on third - managing batch size, beginning with matching it to team velocity.

Beyond improving team productivity, reducing batch size delivers several other benefits:

- Reducing cycle time - smaller deliverables can be delivered & feedback harvested, faster.
- Reducing delivery risk - smaller changes have lower impact & lower risk if something goes wrong.
- Reducing integration risk - small, continuously integrated & tested changes surface integration issues faster & make them easier to isolate & fix.
- Reducing architectural complexity - need to deliver in small batches forces apps toward decoupled, modular architectures rather than

large monoliths.

- Improving testing & quality - QA can run tests more frequently on smaller change sets, catching defects sooner with less impact.
- Reducing waste - Waste is easier to identify & address in small changes than buried within large ones.

Sr. No. Date

Topic

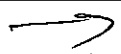
Sign

4

Tuning & Continuous

Improvement

Q. 1. Explain the core themes that form basis of multiple innovation-centric plays.



1. Achieving Multi-Speed IT

Supporting many delivery pipelines with varying cycle times & velocities is a reality. Most business sys depend on services from many apps. Coordination & traceability across these pipelines is essential.

2. Building Right Thing

Drawing on Lean Startup movement & Design Thinking this theme ensures effort isn't wasted building features no one wants.

3. Enabling Experimentation

Innovation need rapid, structured experimentation using MVPs & "fail fast" - rather than building fully functional products to find out if they're right.

4. Delivering Antifragile Systems

Innovation-edge apps must go beyond mere resilience to antifragility; grace at scale, with complex distributed architectures, failures are inevitable & must make system stronger, not just keep it standing.

Q. 2.

Explain various cloud consumption models that help determine what components of environment are

→ platform will exist on & need to support.
A cloud-hosted environment, delivered as IaaS, PaaS or containers - provides environment abstraction needed for an antifragile DevOps Platform. Cloud Consumption models

Public Cloud:

Located in vendor's datacenter, managed by vendor & multi-tenant. This was original cloud model & is still default mental image of "the cloud".

Private Cloud:

A self-managed cloud running on org's own hardware within its own datacenter & firewall. Traditionally chosen for mission-critical, highly regulated apps.

Dedicated Cloud:

A newer model of a single-tenant, vendor-managed cloud, but hosted off-premises in vendor's own datacenter on vendor's hardware.

Local Cloud:

Delivers "best of both": cloud runs in client's own datacenter but is remotely managed by vendor through a tether tech, privately managed, on-premises Cloud.

Hybrid Cloud:

Most large org. combine several of above models - public, private, dedicated, &/or local - along with traditional on-premise hardware.

3. Explain new fluid guidelines & principles within DevOps teams.

→ Its Enterprise Architecture (EA) is incorporated into DevOps teams, org. must maintain delivery pace without chaos, meaning limiting architectural over-engineering & providing only min. flexible set of EA guidance rather than rigid edicts, so devs can still quickly identify critical design issues without being slowed down. Fluid principles:

- Avoid substandard/unsupported materials - since development uses both commercial & open-source software across on-premise & cloud infra; teams must treat app. support prioritarily. EA must work with development to spot where constraints lead to testing shortcuts that cause compliance/quality issues.

- New tech. shouldn't rehouse old problems - architects should avoid rigid mandates forcing new tech into unresolved technical debt simply extending life of underlying problem instead of clearing it.

- Monitor & manage sub-contractors - whichever cloud model is adopted, abstracting infra or app stack should not mean losing visibility & control over performance & end-user experience.

To operationalise these principles, org. are asked to take actions to embed EA sustainably into DevOps programs:

Comms - explain why flexible architecture practices & standards matter (e.g. why API instrumentation then improves quality) rather than imposing rigid rules

Collabs - apply flexible governance: so small agile teams still consider wider business context, balancing big-picture portfolio strategy with hands-on project-level support from needs through delivery.

Q-4. Explain cumulative benefits of combining multiple tools, with ex

→ While individual DevOps tools can each be justified by their own ROI, real, larger gains highlighted in industry research come from examining and combining tools across software lifecycle.

Ex: App Experience Analytics + Agile Requirement Mgmt + Test Design + Test Data Mgmt

Step 1 - App experience analytics: Tools capture real app usage, functional usage & live performance data, feeding it back to agile teams as input for need changes & iterative development - creating a constant cycle of intelligence that improves app with every release.

- Step 2 - Needs & Test Case Design: As needs evolve based on that analytics feedback, agile needs - design tools automatically generate & execute test case, maximising functional coverage with smallest no. of tests while removing outdated, same or duplicate test cases.

Step 3 - Integrating Test Data Mgmt: When a test data warehouse is integrated to automatically supply synthetic data matched to test cases from Step 2, benefits compound further - improving quality, testing efficiency & reduced costs.

Q 5 Explain various talent mgmt. hacks when selecting DevOps professionals.

→ With demand for DevOps skills outstripping supply, & DevOps needing no formal qualifications - HR must work closely with IT on careful, non-traditional sourcing & screening. Ideal candidates are assessed across 3 skill dimensions: People (evidence of presenting, engaging stakeholders, ability to articulate strategy & collab), Process (experience with Agile approaches like Scrum, Kanban) & Tech (a breadth of skills across SDLC)

Beyond screening, org. can use specific talent mgmt. "hacks" to address supply gap, with imp. caveat that age should not be a factor.

IT Apprenticeship & Graduate Intake Programs
Programs that don't need formal IT qualifications, instead seeking passion for tech. A key risk is that rotating eager apprentices through highly siloed, process-centric change-reversal parts of org can dampen their enthusiasm; bestly mitigated by organising them into self-managing teams with mentors & meaningful work.

DevOps Hackathons & Coding Days
Short, intensive events that instill collab. & team work. To maximise DevOps benefit: balance teams across devs, sysadmins, PM, architects; provide broader deliverables beyond new apps & manage expectations realistically - a hackathon yields alpha-quality output not production-ready code so longer online developer challenges are better suited to producing genuinely viable business prototypes.

Sr.No. Date

5

Topic

Scaling & Leading DevOps
for The Enterprise

Sign

Q.1. Explain in brief core themes designed to scale DevOps adoption across a large, potentially distributed org.

→ Large org has many div., often grown through merge & acquisitions, each with its own platform, maturity, processes & culture, governed by rigid practices that stifle speed & innovation. 4 core themes:

1. Organizational Culture

DevOps is fundamentally a cultural movement - building trust, commy & collab across all stakeholders in delivery pipeline is primary goal, with automation & process improvement as enablers, not ends in themselves. Cultural transformation is never "done" - it must be continuously reinforced, or inertia simply reforms in new culture too.

2. Standardization of Tools & Practices

Continuous Improvement is only possible once process is standardized - an inconsistently executed process cannot be reliably measured/improved. Org. should define a minimal, standardized set of processes & tools from which teams select, while keeping enough flexibility to avoid rigidity that undermines agility.

3. Organized Adoption

DevOps adoption across a large enterprise must be rolled out in an organised staged manner - onboarding sets of projects/teams, one wave at a time, sized to available coaching/enablerment capacity - to minimise productivity dip & use scarce coaching resources efficiently.

4. Breaking Down Organizational Silos

Restructuring reporting lines top-down is rarely feasible/successful in large org. Instead, matrix teams - where practitioners keep their existing reporting line but work functionally within a cross functional team reporting into that team's leadership.

Q.2. List potential questions all stakeholders need to ask continuously to establish a culture of innovation at scale needed for a large distributed org.

→ To scale a culture of innovation across a large distributed org, all stakeholders - from line of business to Dev to Ops - need to be continuously asking q such as:-

- How do I ensure I'm asking right q?
- How can teams keep validating their progress?
- How do I prioritize work in an experimentation focused development model.
- What metrics need to be measured to validate an experiment's success or failure?

- As users fulfill their needs, how do I determine what outcomes the org. is looking for?
- How do I visually communicate the end goals of a certain user experience across all components being delivered?
- How are duties & responsibilities distributed across teams & stakeholders?
- Who owns the overall vision for what needs to be delivered, while individual teams run lower-granularity experiments?
- As users interact with business via app/service, what user experience do I want them to have?
- How can I break these experiments down into min. viable products/features deliverable with min. investment, for rapid feedback?
- What is next set of goals to deliver to keep moving forward as experiments are run?

Q. 3. How do you develop a scalable team model for DevOps?
What are parameters needed in these cross-functional stakeholders?

→ A scalable DevOps team-model is built around small, cross-functional teams of stakeholders that can be replicated & connected at scale satisfying the following parameters:

- All cross-functional skills needed for work are available to team, via dedicated members or shared resources.

- Team members can communicate & collaborate without organizational silos disrupting their ability to work together.
- They have visibility into each other's work.
- They can pass artifacts back & forth across an integrated toolchain.
- Team is small enough to foster agility & independence.
- Model is scalable to large, distributed teams by replicating the small-team unit in large nos.
- Sets of these teams can communicate & collaborate with each other requiring integrated tools & standardized processes.
- Overarching governance models manage teams at scale while still allowing individual teams autonomous agility.
- App/sys architecture supports independent development & delivery of components by individual teams, which are continuously integrated & tested.
- Team model is repeatable to assemble larger structures & supports geographically distributed teams.

Q.6.

Explain reasons that apply to DevOps transformation in context of driving cloud adoption.

→ Any transformation effort needs a compelling, catalysing reason to act - without one, orgs recognise need for change but fail to act on it.

In specific context of driving cloud adoption the commonest compelling reasons to act on a DevOps transformation include:

- Merger, acquisition & divestiture:- M&A bring in new people, apps, sys. tools, platforms & cultures that need to be integrated & standardized; divestiture creates an opportunity to revisit & optimize what remains.
- Unexpected service interruption:- A high-profile outage puts the org. into recovery mode & creates urgency to ensure it never recurs frequently by adopting more resilient, cloud-based DevOps platforms & automated delivery practices.
- Launch of an innovation initiative:- Whether prompted by a competitor, a market shift, or a new executive, launching innovation initiatives is a natural opportunity to build out the "innovation edge", which in turn typically needs cloud platforms to support rapid experimentation, MVPs & scaling.
- Enhancing IT Productivity:- Where priority is significantly improving productivity of existing IT org, optimization focused DevOps plays are needed & cloud adoption is frequently the enabling layer for that optimization.

Q.5 Explain goal of each pilot project. Explain criteria to be considered when selecting pilot projects.

Goal of Each Pilot Project:

- Validate that capability being adopted is delivering targeted results.
 - Determine lessons learnt from adopting the capability.
 - Validate that approach & method used to adopt capability is right one.
- DevOps capabilities identified in adoption roadmap should be validated via experiments using pilot projects - one project per capability being adopted, so each capability effect can be isolated.

Criteria for Selecting Pilot Projects:

Pilot project chosen should be good exemplar of typical project in org - Selection criteria include:

- Technology Stack - widely adopted across org or unique to this project?
- Platform deployed - Is it deployment platform an organizational standard or a one-off?
- Team location - Is team co-located / distributed matching org's norm?
- Team composition - Is it a typical mix of employees / contractors & veterans / new hires?